本节内容：

- CSS选择器
- 文本样式设置

一、基础认知

1.1 CSS的介绍

- CSS：层叠样式表（Cascading style sheets）
- CSS作用是什么？？
  - 给页面中的HTML标签设置样式

结构：HTML（决定了身体）　表现：CSS（决定了样式美观）　行为：JavaScript（决定了交互的动态效果）

1.2 CSS写法规则

- 写在哪里？
  - css写在style标签中，style标签一般写在head标签里面，title标签下面，你也可以把它放在HTML文档的 任何地方。

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
  选择器
    p {
        css属性
      color: red;
    }
      属性名      属性值
  </style>
</head>
<body>
  <p>hello world 你好世界</p>
</body>
</html>
```

1.3 CSS初体验

常见CSS属性：

css常见属性	作用
color	文字颜色
font-size	字体大小
background-color	背景颜色
width	宽度
height	高度

代码示例：

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    p {
      color: red;
      font-size: 30px;
      background-color: pink;
      width: 300px;
      height: 300px;
    }
  </style>
</head>
<body>
  <p>hello world 你好世界</p>
</body>
</html>
```

> 注意：
>
> 1. CSS标点符号都是 `英文状态` 下的
> 2. 每个样式都是 `键值对` 的形式，写完之后最后要以分号 `;` 结尾

1.4 CSS引入方式

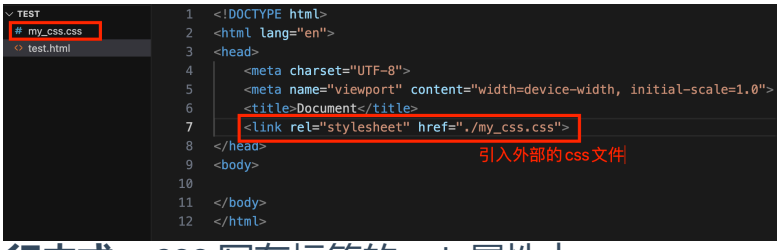
- **内嵌式**：CSS 写在style标签中
  - 提示：style标签虽然可以写在页面任意位置，但是通常约定写在 `head` 标签中

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <style>
        /* CSS代码 */
    </style>
</head>
<body>

</body>
</html>
```

- **外联式**：CSS 写在一个单独的 `.css` 文件中

- 需要通过 `link标签` 在网页中引入。
- **rel属性**：表示的是当前html与被导入文件之间的关系
- **href属性**：是文档链接地址。



- **行内式**：CSS 写在标签的style属性中

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-sca
    <title>Document</title>
</head>
<body>
    <h1 style="color: red;">一级标题</h1>
</body>
</html>
```

小结： CSS常见三种引入方式的特点区别有哪些（书写位置、作用范围、使用场景）？

引入方式	书写位置	作用范围	使用场景
内嵌式	CSS 写在style标签中	当前页面	小案例
外联式	CSS 写在单独的css文件中，通过link标签引入	多个页面	项目中
行内式	CSS 写在标签的style属性中	当前标签	配合js使用

二、基础选择器

选择器的作用： 为了告诉浏览器你要为哪个标签设置样式。

2.1 标签选择器

结构：

```
标签名 {
  属性名:属性值;
  ...
}
```

作用： 通过标签名，找到页面中所有这类标签，设置样式

> **注意点：**
>
> 1. 标签选择器选择的是一类标签，而不是单独某一个
> 2. 标签选择器无论嵌套关系有多深，都能找到对应的标签

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="widt
    <title>Document</title>
    <style>
        p {
            color: red;
        }
    </style>
</head>
<body>
    <p>标签选择器</p>
</body>
</html>
```

2.2 类选择器

结构：

```
.类名 {
  属性名:属性值;
  ...
}
```

作用： 通过类名，找到页面中所有带有这个类名的标签，设置样式

如何设置给标签设置类名？

所有标签上都有 `class` 属性， `class` 属性的属性值称为类名（类似于名字）

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <title>Document</title>
    <style>
        .test{
```

```
            background-color: pink;
        }

        .test2{
            color: red;
        }
    </style>
</head>
<body>
    <h1>hello</h1>
    <div>
        <!-- 每个标签都有class属性，用于设置标签的类名 -->
        <p class="test">这是一个段落</p>
    </div>
    <!-- 可以同时设置多个类名，每个类名之间用空格隔开 -->
    <h2 class="test test2">这是一个标题</h2>
</body>
</html>
```

> **注意点：**
>
> 1. 类名可以由数字、字母、下划线、中划线组成，但不能以数字或者中划线开头
> 2. 一个标签可以同时有多个类名，类名之间以空格隔开
> 3. 类名可以重复，一个类选择器可以同时选中多个同类名的标签

2.3 id 选择器

结构：

```
#id名 {
  属性名:属性值;
  ...
}
```

作用： 通过id属性值，找到页面中带有这个id属性值的标签，设置样式

如何设置给标签设置id名？

所有标签上都有 `id` 属性， `id` 属性的属性值称为 `id` 名（类似于身份证号）

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
```

```html
    <title>Document</title>
    <style>
        #test{
            background-color: pink;
        }

        #test2{
            color: red;
        }
    </style>
</head>
<body>
    <h1>hello</h1>
    <div>
        <!-- 每个标签都有id属性，用于设置标签的id名 -->
        <p id="test">这是一个段落</p>
    </div>
    <h2 id="test2">这是一个标题</h2>
</body>
</html>
```

> **注意点：**
>
> 1. id属性值类似于身份证号码，在一个页面中是唯一的，不可重复的！
> 2. 一个标签上只能有一个id属性值
> 3. 实际情况中id可以重复，但这违反了规范。

2.4 `class` 与 `id` 的区别

- `class` 类名与 `id` 属性值的区别
  - `class` 类名相当于姓名，可以重复，一个标签可以同时有多个 `class` 类名
  - `id` 属性值相当于身份证号码，不可重复，一个标签只能有一个 `id` 属性值
- `类选择器` 与 `id选择器` 的区别
  - `类选择器` 以 `.` 开头
  - `id选择器` 以 `#` 开头
- 实际开发的情况
  - 类选择器用的最多
  - `id` 一般配合 `js` 使用，除非特殊情况，否则不要使用 `id` 设置样式

2.5 通配符选择器

结构：

```
* {
  属性名:属性值;
  ...
}
```

作用： 找到页面中所有的标签，设置样式。一般用于去除标签默认的 `margin` 和 `padding` 从而方便后面的布局。

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <title>Document</title>
    <style>
        *{
            padding: 0;  /* 去除标签的内边距 */
            margin: 0;  /* 去除标签的外边距 */
            background-color: pink;
        }
    </style>
</head>
<body>
    <h1>hello</h1>
    <div>
        <p>这是一个段落</p>
    </div>
    <h2>这是一个段落</h2>
</body>
</html>
```

2.6 后代选择器

结构：

```
A B{
  属性名:属性值;
  ...
}
```

作用： 可以选中 `A` 的所有后代 `B` 。（这里的 `A` 和 `B` 可以是类名、id名等等，中间的空格才是后代的意思）

```
<!DOCTYPE html>
<html lang="en">
```

```
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <title>Document</title>
    <style>
        /* 选中.box的所有后代p标签 */
        .box p {
            background-color: pink;
        }
    </style>
</head>

<body>
    <h1>hello</h1>
    <div class="box">
        <p>这是一个段落</p>
        <p>这是一个段落</p>
        <div>
            <p>这是一个段落</p>
            <p>这是一个段落</p>
        </div>
    </div>
    <p>这是一个段落</p>
    <p>这是一个段落</p>
</body>

</html>
```

2.7 子代选择器

结构：

```
A>B{
  属性名:属性值;
  ...
}
```

作用： 可以选中 `A` 的子代 `B`。（这里的 `A` 和 `B` 可以是类名、id名等等，中间的 `>` 才是子代的意思。

```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <title>Document</title>
    <style>
        /* 选中.box的子代p标签 */
        .box>p {
            background-color: pink;
```

```
        }
    </style>
</head>

<body>
    <h1>hello</h1>
    <div class="box">
        <p>这是一个段落</p>
        <p>这是一个段落</p>
        <div>
            <p>这是一个段落</p>
            <p>这是一个段落</p>
        </div>
    </div>
    <p>这是一个段落</p>
    <p>这是一个段落</p>
</body>

</html>
```

2.8 伪类选择器

作用：用于选中处于 `特定状态` 或 `特定位置` 的元素，而不是根据它们的名称、属性或关系来选择。伪类选择器以冒号 `:` 开头，通常附加在选择器的末尾。

常见的伪类选择器包括但不限于：

1. `:hover`：选中鼠标悬停在元素上时的状态。

   ```
   a:hover {
       /* 设置a标签悬浮时的样式 */
   }
   ```

2. `:active`：选中被用户激活（例如被点击）的元素。

   ```
   button:active {
       /* 设置button标签点击时的样式 */
   }
   ```

3. `:focus`：选中当前拥有焦点的元素，例如表单元素。

   ```
   input:focus {
       /* 设置input标签聚集时的样式 */
   }
   ```

4. `:first-child`：选中某个父元素下的第一个子元素。

```
/* 选中ul子代标签中的第一个li*/
ul>li:first-child {
    /* CSS 样式 */
}
```

5. `:last-child`：选中某个父元素下的最后一个子元素。

```
/* 选中ul子代标签中的最后一个li*/
ul>li:last-child {
    /* CSS 样式 */
}
```

6. `:nth-child()`：根据指定的数字选中某个父元素下的第几个子元素。

```
/* 选中ul子代标签中的第3个li*/
ul>li:nth-child(3) {
    /* CSS 样式 */
}
```

2.9 CSS选择器权重

CSS选择器优先级遵循以下核心原则：

1. 权重值大者优先
2. 相同权重值时后定义者优先
3. `!important` 具有最高权重（慎用）

2.9.1 权重十进制表

选择器类型	示例	十进制权重
内联样式	`<div style="color:red;">`	1000
ID选择器	`#header`	100
类选择器	`.active`	10
伪类	`:hover`	10
元素选择器	`div`	1
伪元素	`::before`	1
通配符	`*`	0
`!important`	`color: red !important;`	权重无限大（∞）

2.9.2 复合选择器权重计算示例

选择器	组成分解	十进制权重计算	总权重
`div`	1个元素选择器	1	1
`.container`	1个类选择器	10	10
`#main`	1个ID选择器	100	100
`div.active`	1个元素 + 1个类	1 + 10	11
`#header .menu`	1个ID + 1个类	100 + 10	110
`ul li a:hover`	3个元素 + 1个伪类	1×3 + 10	13
`#box .box2 .box3 #box4`	2个ID + 2个类	100×2 + 10×2	220
`#sidebar > .item:first-child`	1个ID + 1个类 + 1个伪类	100 + 10 + 10	120
`body.home #content .post:nth-child(2)`	1个元素 + 1个ID + 2个类 + 1个伪类	1 + 100 + 10×2 + 10	131

2.10 颜色常见取值

使用场景： 设置文字颜色、背景颜色、边框颜色。

属性值：

颜色表示方式	表示含义	属性值
关键词	预定义的颜色名	red, green, blue, yellow.....
rgb表示法	红绿蓝三原色。每项取值范围：0~255	rgb(0,0,0), rgb(255,255,255), rgb(255,0,0).....
rgba表示法	红绿蓝三原色+a表示透明度，取值范围是0~1	rgba(255,255,255,0.5), rgba(255,0,0,0.3).....
十六进制表示法	#开头，将数字转换成十六进制表示	#000000、#ff0000、#e92322，简写：#000、#f00

三、字体和文本样式

3.1 字体大小

属性名： `font-size`

取值： 数字 + px

> **注意点：**
>
> 谷歌浏览器默认文字大小是 `16px`
>
> 单位需要设置，否则无效

3.2 字体粗细

属性名： `font-weight`

取值：

- 关键字：

正常	normal
加粗	bold

- 纯数字：100~900的整百数：

正常	400
加粗	700

> **注意点：**
>
> 1. 不是所有字体都提供了九种粗细，因此部分取值页面中无变化
> 2. 实际开发中以：正常、加粗两种取值使用最多
> 3. 正常字体粗细为400

3.3 文本缩进

属性名： `text-indent`

取值：

- 数字+px
- 数字+em（推荐：1em = 当前标签的font-size的大小）

最初的现代计算机并不容易操作。当时的编程实际上是手工将电线连接成一排排电路来实现的。后来出现了机器语言和汇编语言，允许用户用代码为计算机编程，但这两种语言都需要对计算机的架构有深入的了解，使得许多科学家难以掌握。

20世纪50年代，随着符号语言的发展，特别是由约翰·巴克斯及其团队在加州圣何塞的IBM开发的“公式翻译”语言Fortran，这种情况发生了变化。利用Fortran，用户可以用人类可读的指令来编程，例如x = 3 + 5。然后由编译器将这些指令转换成快速、高效的机器代码。

3.4 文本水平对齐方式

属性名： text-align

取值：

属性值	效果
left	左对齐
center	居中对齐
right	右对齐

> **注意点：**
>
> 如果需要让文本水平居中，text-align属性给文本所在标签（文本的父元素）设置

3.5 文本修饰

属性名： text-decoration

取值：

值	作用	示例
`none`	无装饰线（默认值）	清除超链接下划线
`underline`	下划线	重点标注
`overline`	上划线	标题装饰
`line-through`	删除线	价格打折效果

- 最新研究：地球的“死亡速度”远超出科学家预期
- 我们已处在第6次物种大灭绝边缘？
- 为什么早期人类要离开非洲大陆？
- 法国启动量子技术国家战略 计划5年内投资18亿欧元
- “已读不回”和“无人点赞”真的让我很焦虑
- 全球疫苗开打，谁将率先获得群体免疫？
- 能逃避抗体攻击的新冠变异株出现，新冠疫苗还有效吗？

> **注意点：**
>
> 开发中会使用 `text-decoration: none;` 清除a标签默认的下划线

3.6 行高

作用： 控制一行的上下行间距

属性名： line-height

取值：

- 数字+px

应用： 让单行文本垂直居中可以设置 `line-height: 文字父元素高度`

3.7 行内元素特性

标签元素分类：

- 块级标签：独占一行
- 行内标签：不会换行

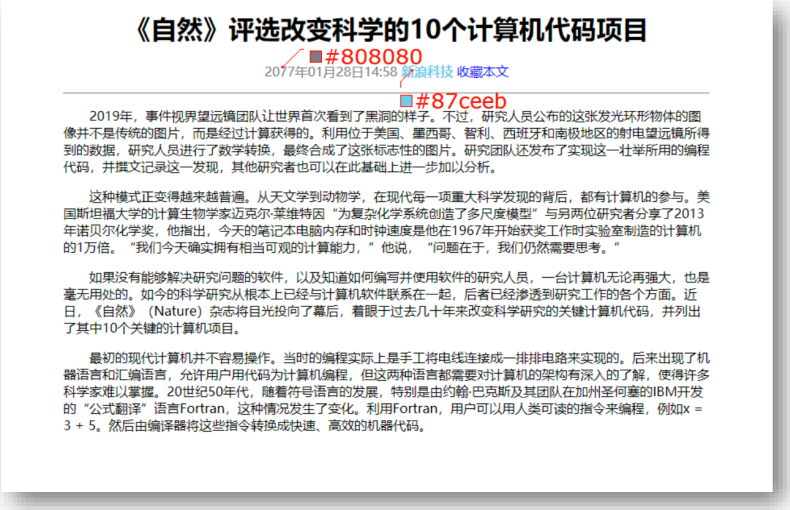
> **注意：**
>
> 1. 在使用 `text-align: center;` 居中标签内容的文本时，标签内的 `行内元素` 也会被当做成 `文本` 居中，而块级标签不会收到影响

> 2. 如果想要让块级标签水平居中，可以在块级标签上设置 `margin: 0 auto;` 样式，但前提是块级元素有设置宽度，否则没效果。

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <title>Document</title>
    <style>
        .big{
            text-align: center;
        }
        .p1{
            width: 100px;
            background-color: red;
            margin: 0 auto;  /* 居中 */
        }
    </style>
</head>
<body>
    <div class="big">
        <a href="#">行内标签</a>
        <p class="p1">块级标签</p>
        文本
    </div>
</body>
</html>
```

四、练习

新闻网页案例-效果图



文本素材

《自然》评选改变科学的10个计算机代码项目

2077年01月28日14:58 新浪科技 收藏本又

2019年，事件视界望远镜团队让世界首次看到了黑洞的样子。不过，研究人员公布的这张发光环形物体的图像并不是传统的图片，而是经过计算获得的。利用位于美国、墨西哥、智利、西班牙和南极地区的射电望远镜所得到的数据，研究人员进行了数学转换，最终合成了这张标志性的图片。研究团队还发布了实现这一壮举所用的编程代码，并撰文记录这一发现，其他研究者也可以在此基础上进一步加以分析。

这种模式正变得越来越普遍。从天文学到动物学，在现代每一项重大科学发现的背后，都有计算机的参与。美国斯坦福大学的计算生物学家迈克尔•莱维特因“为复杂化学系统创造了多尺度模型”与另两位研究者分享了2013年诺贝尔化学奖，他指出，今天的笔记本电脑内存和时钟速度是他在1967年开始获奖工作时实验室制造的计算机的1万倍。“我们今天确实拥有相当可观的计算能力，”他说，“问题在于，我们仍然需要思考。”

如果没有能够解决研究问题的软件，以及知道如何编写并使用软件的研究人员，一台计算机无论再强大，也是毫无用处的。如今的科学研究从根本上已经与计算机软件联系在一起，后者已经渗透到研究工作的各个方面。近日，《自然》（Nature）杂志将目光投向了幕后，着眼于过去几十年来改变科学研究的关键计算机代码，并列出了其中10个关键的计算机项目。

最初的现代计算机并不容易操作。当时的编程实际上是手工将电线连接成一排排电路来实现的。后来出现了机器语言和汇编语言，允许用户用代码为计算机编程，但这两种语言都需要对计算机的架构有深入的了解，使得许多科学家难以掌握。20世纪50年代，随着符号语言的发展，特别是由约翰•巴克斯及其团队在加州圣何塞的IBM开发的“公式翻译”语言Fortran，这种情况发生了变化。利用Fortran，用户可以用人类可读的指令来编程，例如x=3 + 5。然后由编译器将这些指令转换成快速、高效的机器代码。